

Tailored Limb Counts, Faster Arithmetic: Improved TMVP Decompositions for Curve5453 and Curve6071

Murat Cenk¹, N. Gamze Orhon Kılıç², Halil Kemal Taşkın³ and Oğuz Yayla²

¹ Ripple Lab Inc, Canada

² Institute of Applied Mathematics, Middle East Technical University, Türkiye

³ ASELSAN, Türkiye

Abstract. Curve5453 and Curve6071 are Montgomery curves over the primes $2^{545} - 3$ and $2^{607} - 1$, providing 271- and 302-bit classical security, respectively. Their TMVP-based field multiplication in 10-limb representation costs 77 multiplications. We reduce this to 60 for Curve5453 (22% fewer) using a 9-limb radix- 2^{61} representation, and to 54 for Curve6071 (30% fewer) using a 12-limb radix- 2^{51} representation with hierarchical block-level TMVP. Choosing the limb count to produce 3×3 Toeplitz blocks aligns the structure with the size-3 TMVP formula, computing each block product in 6 multiplications rather than 9. Portable C implementations benchmarked on ARM64 and x86-64 confirm speedups of up to 16% in field multiplication and 13% in scalar multiplication. On ARM64, Curve5453 reaches 90.6% of OpenSSL’s assembly-optimized NIST P-521 ECDH throughput with 12 additional bits of classical security, and Curve6071 delivers 302-bit classical security at 80.8% of P-521’s throughput.

Keywords: TMVP · Field multiplication · Montgomery curves · Radix representations · Efficient implementation

1 Introduction

Elliptic curve cryptography (ECC), introduced independently by Miller [Mil86] and Koblitz [Kob87], is widely deployed in TLS, SSH, and related protocols. Widely used curves such as Curve25519 [Ber06] and NIST P-256 [Nat23] target 128-bit classical security; however, the prospect of fault-tolerant quantum computers running Shor’s algorithm [Sho94]—estimated to require roughly 2,124 logical qubits at this security level [HJN⁺20]—motivates higher security margins, particularly for hybrid key exchange protocols that pair classical ECC with post-quantum primitives. The SafeCurves project [BL25] evaluates curves against criteria spanning parameter generation, ECDLP hardness, and implementation-level security; yet among compliant curves, none beyond 260-bit security has demonstrated competitive performance.

Montgomery curves [Mon87, CS18] provide inherent constant-time scalar multiplication via the Montgomery ladder [BL17], making them well suited for side-channel-resistant implementations. The dominant cost in ladder-based scalar multiplication is field multiplication, and for primes of the form $p = 2^m - c$ —Mersenne ($c = 1$) or Crandall primes [Cra92]—the combined multiplication-and-reduction step can be cast as a Toeplitz matrix-vector product (TMVP). Ali and Cenk [AC18] first exploited this structure for $\mathbb{F}_{2^{521}-1}$; Taşkın and Cenk [TC18, TC20] generalized the approach to Curve25519 and introduced the notion of *TMVP-friendly primes*, where the reduction constant admits efficient scaling.

Building on this line of work, Cenk et al. [COKTY25] proposed two Montgomery curves $E_M: y^2 = x^3 + Ax^2 + x$ over TMVP-friendly primes with high classical security: *Curve5453*, over the Crandall prime $p = 2^{545} - 3$ with $A = 115794$ and 271-bit security; and *Curve6071*, over the Mersenne prime $p = 2^{607} - 1$ with $A = 2720010$ and 302-bit security—the highest among SafeCurves-compliant curves. Both curves have cofactor 4, coefficient $B = 1$, and are generated by a rigid, deterministic process; Curve6071 satisfies all eleven SafeCurves criteria, while Curve5453 meets all except completeness. Using 10-limb radix representations (radix- 2^{55} for $\mathbb{F}_{5453}$, radix- 2^{61} for $\mathbb{F}_{6071}$), field multiplication was realized via recursive TMVP decomposition of the 10×10 Toeplitz matrices at a cost of 77 multiplications, with variable-base scalar multiplication measured at 871,898 and 903,542 cycles on ARM64, respectively.

In this work, we investigate alternative limb representations for the fields in [COKTY25]. For $\mathbb{F}_{5453}$, a 9-limb radix- 2^{61} representation yields a 9×9 Toeplitz matrix that admits a 3×3 block decomposition; applying the size-3 TMVP formula at the inner level reduces the cost to 60 multiplications. For $\mathbb{F}_{6071}$, a 12-limb radix- 2^{51} representation produces a 12×12 Toeplitz structure supporting hierarchical 3×3 block TMVP at both outer and inner levels, achieving 54 multiplications. The reduction from 77 to 60 multiplications for $\mathbb{F}_{5453}$ (22%) arises because the 9-limb representation yields a 3×3 block partition whose sub-matrices align directly with the size-3 TMVP formula, whereas the 10-limb baseline requires a less natural recursive $2 \times (2 + 3)$ decomposition. For $\mathbb{F}_{6071}$, we systematically explore four strategies spanning 104 to 54 multiplications; the best exploits the $12 = 3 \times 4$ factorization to apply a size-3 TMVP at the block level, reducing the number of 4×4 sub-products from nine to six and achieving a 30% reduction. Beyond the algebraic savings, fewer limbs in the 9-limb case also reduce the per-operation cost of field additions and carry propagation throughout the Montgomery ladder, compounding the benefit at the scalar multiplication level. We implement all strategies in portable C and provide dual-platform benchmarks on ARM64 and x86-64—the first cross-platform evaluation for these curves—together with a comparative ECDH throughput analysis against OpenSSL.

The remainder of this paper is organized as follows: Section 2 recalls the TMVP formulas and introduces the block decomposition framework; Section 3 develops the alternative representations and multiplication strategies for both fields; Section 4 presents dual-platform benchmarks and a comparative ECDH throughput analysis; and Section 5 concludes.

2 Preliminaries and Notation

This section establishes the notation and algebraic tools used throughout the paper: radix representations of field elements, the Toeplitz structure of the resulting multiplication matrices, the TMVP formulas that reduce the cost of computing these products, and a block decomposition framework that organizes the computation at two levels.

2.1 Radix Representation

Let $p = 2^m - c$ for a small integer $c > 0$, and let $\alpha \in \mathbb{F}_p$. A radix- 2^r representation splits α into $t = \lceil m/r \rceil$ limbs, each fitting in a single machine word:

$$\alpha = \sum_{i=0}^{t-1} \alpha_i 2^{\lceil ri \rceil}, \quad 0 \leq \alpha_i < 2^r. \quad (1)$$

Viewing α as the polynomial $\alpha(x) = \sum_i \alpha_i x^i$ evaluated at $x = 2^r$, the product $\gamma = \alpha \cdot \beta \bmod p$ corresponds to polynomial multiplication modulo $x^t - d$, where $d = 2^{r(t-m)} \cdot c$, reducing upper coefficients via $\alpha_i x^{t+j} \equiv d \cdot \alpha_i x^j \pmod{x^t - d}$.

2.2 Toeplitz Matrices

A Toeplitz matrix is an $n \times n$ matrix with $T_{i,j} = t_{i-j}$, determined by $2n - 1$ parameters $(t_{-(n-1)}, \dots, t_0, \dots, t_{n-1})$. The polynomial multiplication from Section 2.1 yields precisely such a structure: a $t \times t$ Toeplitz matrix whose lower triangle contains the original limbs and whose upper triangle entries are scaled by d . This matrix admits a natural decomposition $M = L + d \cdot U$, where L is the lower triangular part and U is the strictly upper triangular part; the products $L\beta$ and $U\beta$ are computed independently and combined as $\gamma = L\beta + d \cdot U\beta$.

2.3 Toeplitz Matrix-Vector Product

Computing $T \cdot v$ by the schoolbook method costs n^2 multiplications. The TMVP technique reduces this via Karatsuba-like decompositions that exploit the diagonal structure; we recall the formulas for sizes 2 and 3.

2.3.1 TMVP of Size 2.

Given a 2×2 Toeplitz matrix with parameters $(\alpha_0, \alpha_1, \alpha_2)$ and a vector $(\beta_0, \beta_1)^\top$, the product is computed as

$$\begin{bmatrix} \alpha_0 & \alpha_1 \\ \alpha_2 & \alpha_0 \end{bmatrix} \cdot \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = \begin{bmatrix} P_1 + P_2 \\ P_1 + P_3 \end{bmatrix} \quad (2)$$

where $P_1 = \alpha_0(\beta_0 + \beta_1)$, $P_2 = (\alpha_1 - \alpha_0)\beta_1$, and $P_3 = (\alpha_2 - \alpha_0)\beta_0$. The cost is $3\mathbf{M} + 3\mathbf{A} + 1\mathbf{A}_d^1$, compared with $4\mathbf{M} + 2\mathbf{A}_d$ for the schoolbook method.

2.3.2 TMVP of Size 3.

For a 3×3 Toeplitz matrix with parameters $(\alpha_0, \alpha_1, \alpha_2, \alpha_3, \alpha_4)$ and a vector $(\beta_0, \beta_1, \beta_2)^\top$, the product is computed with six multiplications:

$$\begin{bmatrix} \alpha_0 & \alpha_1 & \alpha_2 \\ \alpha_3 & \alpha_0 & \alpha_1 \\ \alpha_4 & \alpha_3 & \alpha_0 \end{bmatrix} \cdot \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} P_3 + P_4 + P_6 \\ P_2 - P_4 + P_5 \\ P_1 - P_2 - P_3 \end{bmatrix}, \quad \begin{aligned} P_1 &= (\alpha_4 + \alpha_3 + \alpha_0) \cdot \beta_0, \\ P_2 &= \alpha_3 \cdot (\beta_0 - \beta_1), \\ P_3 &= \alpha_0 \cdot (\beta_0 - \beta_2), \\ P_4 &= \alpha_1 \cdot (\beta_1 - \beta_2), \\ P_5 &= (\alpha_0 + \alpha_1 + \alpha_3) \cdot \beta_1, \\ P_6 &= (\alpha_0 + \alpha_1 + \alpha_2) \cdot \beta_2. \end{aligned} \quad (3)$$

The cost is $6\mathbf{M} + 8\mathbf{A} + 6\mathbf{A}_d$, compared with $9\mathbf{M} + 6\mathbf{A}_d$ for the schoolbook method. Applying (2) at the block level with 2×2 sub-matrices yields a size-4 TMVP at $3 \times 3 = 9\mathbf{M}$; recursive composition of (2) and (3) produces TMVPs of arbitrary size.

2.4 Block Decomposition Framework

When $t = nk$ for positive integers n and k , the $t \times t$ Toeplitz matrix M partitions into an $n \times n$ block Toeplitz array of $k \times k$ Toeplitz sub-matrices, with the input vector split into sub-vectors $\mathcal{V}_0, \dots, \mathcal{V}_{n-1}$ of dimension k .

2.4.1 The $L + dU$ Decomposition.

Under the block partition, L is block lower triangular and U is block upper triangular (with strictly upper triangular diagonal blocks), giving

$$\gamma = L \cdot \beta + d \cdot (U \cdot \beta). \quad (4)$$

¹Throughout this paper, $\mathbf{M}$ denotes a multiplication, $\mathbf{A}$ a single-precision addition or subtraction, and $\mathbf{A}_d$ a double-precision addition.

The factor d is applied once to $U\beta$ —typically via bit shifts—rather than embedded in every upper-triangle multiplication. Three types of $k \times k$ sub-matrices arise: diagonal blocks of L are *lower triangular* Toeplitz (reduced cost), off-diagonal blocks are *full* Toeplitz, and diagonal blocks of U are *strictly upper triangular* Toeplitz (cheapest). This decomposition is not required by all methods: when d is amenable to fast scaling, one may retain the full block Toeplitz structure and apply TMVP directly at the block level.

2.4.2 Composition Strategies.

Two computation levels arise: the *outer level* organizes the block products either by schoolbook enumeration or by applying a TMVP formula at the block level (e.g., size-3 TMVP reduces products from 9 to 6); the *inner level* computes each $k \times k$ sub-product by schoolbook (k^2 multiplications) or recursive TMVP. The choice at each level is independent; Section 3 explores combinations for the specific dimensions arising from the radix-2⁶¹ and radix-2⁵¹ representations.

3 Arithmetic on $\mathbb{F}_{5453}$ and $\mathbb{F}_{6071}$

This section develops alternative radix representations and multiplication strategies for $\mathbb{F}_{5453}$ and $\mathbb{F}_{6071}$, applying the block decomposition framework of Section 2.4 to the Toeplitz matrices that arise in each case.

3.1 Representing Elements of $\mathbb{F}_{5453}$ with Radix-2⁶¹

Let $\alpha \in \mathbb{F}_{5453}$ where $p = 2^{545} - 3$. The element α is represented using 9 limbs in a radix-2⁶¹ representation: 8 limbs of 61 bits and one top limb of 57 bits. The limb representation and its corresponding polynomial form are:

$$\begin{aligned} \alpha &= [\alpha_0/61, \dots, \alpha_2/61, \alpha_8/57] \\ \alpha(x) &= \sum_{i=0}^8 \alpha_i 2^{\lceil 61i \rceil} x^i = \alpha_0 + \alpha_1 2^{61} x + \alpha_2 2^{122} x^2 + \dots + \alpha_8 2^{488} x^8. \end{aligned} \quad (5)$$

Since $p = 2^{545} - 3$, evaluating $x^9 = (2^{61})^9 = 2^{549}$ modulo p gives

$$x^9 = 2^4 \cdot 2^{545} \equiv 2^4 \cdot 3 = 48 \pmod{p},$$

so the reduction polynomial is $x^9 - 48$: any term $a_i x^{9+j}$ arising from multiplication reduces as $48 \cdot a_i x^j$.

3.2 Hybrid Multiplication over $\mathbb{F}_{5453}$ for 9-Limb Representation

Using the radix-2⁶¹ representation, field element multiplication over $\mathbb{F}_{5453}$ can be performed by the following matrix-vector multiplication:

$$\begin{bmatrix} \alpha_0 & 48\alpha_8 & 48\alpha_7 & 48\alpha_6 & 48\alpha_5 & 48\alpha_4 & 48\alpha_3 & 48\alpha_2 & 48\alpha_1 \\ \alpha_1 & \alpha_0 & 48\alpha_8 & 48\alpha_7 & 48\alpha_6 & 48\alpha_5 & 48\alpha_4 & 48\alpha_3 & 48\alpha_2 \\ \alpha_2 & \alpha_1 & \alpha_0 & 48\alpha_8 & 48\alpha_7 & 48\alpha_6 & 48\alpha_5 & 48\alpha_4 & 48\alpha_3 \\ \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & 48\alpha_8 & 48\alpha_7 & 48\alpha_6 & 48\alpha_5 & 48\alpha_4 \\ \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & 48\alpha_8 & 48\alpha_7 & 48\alpha_6 & 48\alpha_5 \\ \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & 48\alpha_8 & 48\alpha_7 & 48\alpha_6 \\ \alpha_6 & \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & 48\alpha_8 & 48\alpha_7 \\ \alpha_7 & \alpha_6 & \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & 48\alpha_8 \\ \alpha_8 & \alpha_7 & \alpha_6 & \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 \end{bmatrix} \cdot \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \beta_3 \\ \beta_4 \\ \beta_5 \\ \beta_6 \\ \beta_7 \\ \beta_8 \end{bmatrix} = \begin{bmatrix} \gamma_0 \\ \gamma_1 \\ \gamma_2 \\ \gamma_3 \\ \gamma_4 \\ \gamma_5 \\ \gamma_6 \\ \gamma_7 \\ \gamma_8 \end{bmatrix}. \quad (6)$$

The matrix M in (6) admits a 3×3 block partition with $k = 3$ (Section 2.4). The input vector is partitioned into sub-vectors $\mathcal{V}_0 = (\beta_0, \beta_1, \beta_2)^\top$, $\mathcal{V}_1 = (\beta_3, \beta_4, \beta_5)^\top$, $\mathcal{V}_2 = (\beta_6, \beta_7, \beta_8)^\top$. Applying the $L + dU$ decomposition (Section 2.4.1) with $d = 48$, the block partition involves four distinct 3×3 sub-matrices:

$$\mathcal{A} = \begin{bmatrix} a_0 & 0 & 0 \\ a_1 & a_0 & 0 \\ a_2 & a_1 & a_0 \end{bmatrix}, \quad \mathcal{B} = \begin{bmatrix} a_3 & a_2 & a_1 \\ a_4 & a_3 & a_2 \\ a_5 & a_4 & a_3 \end{bmatrix}, \quad \mathcal{C} = \begin{bmatrix} a_6 & a_5 & a_4 \\ a_7 & a_6 & a_5 \\ a_8 & a_7 & a_6 \end{bmatrix}, \quad \mathcal{D} = \begin{bmatrix} 0 & a_8 & a_7 \\ 0 & 0 & a_8 \\ 0 & 0 & 0 \end{bmatrix}.$$

Here $\mathcal{A}$ is lower triangular (diagonal block of L), $\mathcal{B}$ and $\mathcal{C}$ are full Toeplitz (off-diagonal blocks of L), and $\mathcal{D}$ is strictly upper triangular (diagonal block of U), following the classification in Section 2.4.1. The schoolbook outer composition (Section 2.4.2) yields 12 sub-products of size 3: each of the three output sub-vectors requires one product with each of $\mathcal{A}$, $\mathcal{B}$, $\mathcal{C}$, and $\mathcal{D}$.

Using TMVP for 3×3 Products

Each sub-product is computed using the size-3 TMVP (3). Since $\mathcal{A}$ is lower triangular, its Toeplitz parameters are $(\alpha_0, 0, 0, \alpha_1, \alpha_2)$; the product $P_4 = 0$ is eliminated, reducing the cost to $5\mathbf{M} + 5\mathbf{A} + 4\mathbf{A}_d$ per product. The full Toeplitz matrices $\mathcal{B}$ and $\mathcal{C}$ incur the complete formula at $6\mathbf{M} + 8\mathbf{A} + 6\mathbf{A}_d$ each. The strictly upper triangular $\mathcal{D}$ has parameters $(0, \alpha_8, \alpha_7, 0, 0)$; only three products are non-zero, giving $3\mathbf{M} + 2\mathbf{A} + 2\mathbf{A}_d$. In total: $3 \times 5 + 6 \times 6 + 3 \times 3 = 60\mathbf{M} + 69\mathbf{A} + 54\mathbf{A}_d$.

Using Schoolbook for 3×3 Products

Alternatively, each sub-product is computed by direct matrix-vector multiplication. The lower triangular $\mathcal{A}$ requires $1 + 2 + 3 = 6\mathbf{M} + 3\mathbf{A}_d$ per product, the full matrices $\mathcal{B}$ and $\mathcal{C}$ require $9\mathbf{M} + 6\mathbf{A}_d$ each, and the strictly upper triangular $\mathcal{D}$ costs $3\mathbf{M} + 1\mathbf{A}_d$. The total is $3 \times 6 + 6 \times 9 + 3 \times 3 = 81\mathbf{M} + 48\mathbf{A}_d$.

3.2.1 Combining Lower and Upper

The final unreduced result is $h_i = \ell_i + 48 \cdot u_i$ for $i = 0, \dots, 8$, where $48 = 2^5 + 2^4$ is implemented as $(u_i \ll 5) + (u_i \ll 4)$. Since the intermediate u_i values can reach $\sim 2^{125}$ bits—overflowing 128-bit accumulators when shifted—the u vector is carry-propagated first: $u_{i+1} += u_i \gg 61$, $u_i \&= 2^{61} - 1$ for $i = 0, \dots, 7$. After propagation each $u_i < 2^{61}$, so $(u_i \ll 5) < 2^{66}$ and the combining is safe; u_8 may become nonzero from the carry out of u_7 , so all 9 limbs are combined. Carry propagation costs 8 shifts, 8 ANDs, and $8\mathbf{A}_d$; combining costs 18 shifts and $18\mathbf{A}_d$.

3.3 Representing Elements of $\mathbb{F}_{6071}$ with Radix- 2^{51}

Let $\alpha \in \mathbb{F}_{6071}$ where $p = 2^{607} - 1$. It is represented using 11 limbs of 51 bits and one limb of 46 bits, known as the radix- 2^{51} representation. The limb representation and its corresponding polynomial form are:

$$\alpha = [\alpha_0/51, \dots, \alpha_{10}/51, \alpha_{11}/46] \quad (7)$$

$$\alpha(x) = \sum_{i=0}^{11} \alpha_i 2^{\lceil 51i \rceil} x^i = \alpha_0 + \alpha_1 2^{51} x + \alpha_2 2^{102} x^2 + \dots + \alpha_{11} 2^{561} x^{11}.$$

Since $p = 2^{607} - 1$, evaluating $x^{12} = (2^{51})^{12} = 2^{612}$ modulo p gives

$$x^{12} = 2^5 \cdot 2^{607} \equiv 2^5 = 32 \pmod{p},$$

so the reduction polynomial is $x^{12} - 32$: any term $a_i x^{12+j}$ arising from multiplication reduces as $32 \cdot a_i x^j$.

3.4 Multiplication Strategies for 12-Limb Representation over $\mathbb{F}_{6071}$

Using the radix-2⁵¹ representation, field element multiplication over $\mathbb{F}_{6071}$ can be performed by the following matrix-vector multiplication:

$$\begin{bmatrix} \alpha_0 & d\alpha_{11} & d\alpha_{10} & d\alpha_9 & d\alpha_8 & d\alpha_7 & d\alpha_6 & d\alpha_5 & d\alpha_4 & d\alpha_3 & d\alpha_2 & d\alpha_1 \\ \alpha_1 & \alpha_0 & d\alpha_{11} & d\alpha_{10} & d\alpha_9 & d\alpha_8 & d\alpha_7 & d\alpha_6 & d\alpha_5 & d\alpha_4 & d\alpha_3 & d\alpha_2 \\ \alpha_2 & \alpha_1 & \alpha_0 & d\alpha_{11} & d\alpha_{10} & d\alpha_9 & d\alpha_8 & d\alpha_7 & d\alpha_6 & d\alpha_5 & d\alpha_4 & d\alpha_3 \\ \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & d\alpha_{11} & d\alpha_{10} & d\alpha_9 & d\alpha_8 & d\alpha_7 & d\alpha_6 & d\alpha_5 & d\alpha_4 \\ \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & d\alpha_{11} & d\alpha_{10} & d\alpha_9 & d\alpha_8 & d\alpha_7 & d\alpha_6 & d\alpha_5 \\ \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & d\alpha_{11} & d\alpha_{10} & d\alpha_9 & d\alpha_8 & d\alpha_7 & d\alpha_6 \\ \alpha_6 & \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & d\alpha_{11} & d\alpha_{10} & d\alpha_9 & d\alpha_8 & d\alpha_7 \\ \alpha_7 & \alpha_6 & \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & d\alpha_{11} & d\alpha_{10} & d\alpha_9 & d\alpha_8 \\ \alpha_8 & \alpha_7 & \alpha_6 & \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & d\alpha_{11} & d\alpha_{10} & d\alpha_9 \\ \alpha_9 & \alpha_8 & \alpha_7 & \alpha_6 & \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & d\alpha_{11} & d\alpha_{10} \\ \alpha_{10} & \alpha_9 & \alpha_8 & \alpha_7 & \alpha_6 & \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 & d\alpha_{11} \\ \alpha_{11} & \alpha_{10} & \alpha_9 & \alpha_8 & \alpha_7 & \alpha_6 & \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \beta_3 \\ \beta_4 \\ \beta_5 \\ \beta_6 \\ \beta_7 \\ \beta_8 \\ \beta_9 \\ \beta_{10} \\ \beta_{11} \end{bmatrix} = \begin{bmatrix} \gamma_0 \\ \gamma_1 \\ \gamma_2 \\ \gamma_3 \\ \gamma_4 \\ \gamma_5 \\ \gamma_6 \\ \gamma_7 \\ \gamma_8 \\ \gamma_9 \\ \gamma_{10} \\ \gamma_{11} \end{bmatrix} \quad (8)$$

where $d = 32$. The matrix in (8) admits a 3×3 block partition with $k = 4$ (Section 2.4). The input vector is partitioned into sub-vectors $\mathcal{W}_0 = (\beta_0, \beta_1, \beta_2, \beta_3)^\top$, $\mathcal{W}_1 = (\beta_4, \beta_5, \beta_6, \beta_7)^\top$, $\mathcal{W}_2 = (\beta_8, \beta_9, \beta_{10}, \beta_{11})^\top$. The three distinct 4×4 Toeplitz sub-matrices are:

$$\mathcal{F} = \begin{bmatrix} \alpha_0 & 32\alpha_{11} & 32\alpha_{10} & 32\alpha_9 \\ \alpha_1 & \alpha_0 & 32\alpha_{11} & 32\alpha_{10} \\ \alpha_2 & \alpha_1 & \alpha_0 & 32\alpha_{11} \\ \alpha_3 & \alpha_2 & \alpha_1 & \alpha_0 \end{bmatrix}, \quad \mathcal{G} = \begin{bmatrix} \alpha_8 & \alpha_7 & \alpha_6 & \alpha_5 \\ \alpha_9 & \alpha_8 & \alpha_7 & \alpha_6 \\ \alpha_{10} & \alpha_9 & \alpha_8 & \alpha_7 \\ \alpha_{11} & \alpha_{10} & \alpha_9 & \alpha_8 \end{bmatrix}, \quad \mathcal{H} = \begin{bmatrix} \alpha_4 & \alpha_3 & \alpha_2 & \alpha_1 \\ \alpha_5 & \alpha_4 & \alpha_3 & \alpha_2 \\ \alpha_6 & \alpha_5 & \alpha_4 & \alpha_3 \\ \alpha_7 & \alpha_6 & \alpha_5 & \alpha_4 \end{bmatrix}.$$

The block Toeplitz structure is $\begin{bmatrix} \mathcal{F} & 32\mathcal{G} & 32\mathcal{H} \\ \mathcal{H} & \mathcal{F} & 32\mathcal{G} \\ \mathcal{G} & \mathcal{H} & \mathcal{F} \end{bmatrix}$. We explore three strategies using the framework of Section 2.4.

Schoolbook Outer with Size-3 Inner Products

The matrix also admits a 4×4 block partition with $k = 3$, organized as a two-level hierarchy: 2×2 blocks of 6×6 , each subdivided into 2×2 blocks of 3×3 sub-matrices, yielding five distinct blocks $\mathcal{X}_0, \dots, \mathcal{X}_4$. Applying the $L + dU$ decomposition (Section 2.4.1) at the top level with schoolbook composition produces six 6×6 products; expanding each into $\mathcal{X}_i$ -vector sub-products of size 3 and applying (3) gives $\mathcal{X}_0$ (lower triangular) at $5\mathbf{M}$, $\mathcal{X}_1$ – $\mathcal{X}_3$ (full) at $6\mathbf{M}$, and $\mathcal{X}_4$ (strictly upper triangular) at $3\mathbf{M}$, for a total of $2(2 \times 5 + 6) + 2(4 \times 6) + 2(2 \times 3 + 6) = 104\mathbf{M}$. This exceeds the 10-limb baseline of $77\mathbf{M}$, motivating a coarser block partition.

Schoolbook Outer with Size-4 Inner Products

Returning to the 3×3 block partition with $k = 4$, the $L + dU$ decomposition (Section 2.4.1) with schoolbook outer composition (Section 2.4.2) yields nine 4×4 Toeplitz products. Since $\mathcal{F}$ already contains d -scaled entries in its upper triangle, all nine blocks are full 4×4 Toeplitz matrices. Each 4×4 TMVP is computed by applying (2) at the block level with 2×2 sub-matrices, requiring $3 \times 3 = 9\mathbf{M}$. The total is $9 \times 9 = 81\mathbf{M}$. A variant, denoted `sbtmvp212-51`, applies the size-2 TMVP only at the base level of each 4×4 inner product rather than recursively; it is included in the benchmarks of Section 4.

Block-Level TMVP

The 3×3 block structure is itself Toeplitz at the block level, with parameters

$$T_0 = \mathcal{F}, \quad T_1 = 32\mathcal{G}, \quad T_2 = 32\mathcal{H}, \quad T_3 = \mathcal{H}, \quad T_4 = \mathcal{G}.$$

Applying the size-3 TMVP formula (3) at the block level (recursive TMVP composition, Section 2.4.2) yields six 4×4 products instead of nine:

$$\begin{aligned} P_1 &= (\mathcal{G} + \mathcal{H} + \mathcal{F}) \cdot \mathcal{W}_0, & P_4 &= 32\mathcal{G} \cdot (\mathcal{W}_1 - \mathcal{W}_2), \\ P_2 &= \mathcal{H} \cdot (\mathcal{W}_0 - \mathcal{W}_1), & P_5 &= (\mathcal{F} + 32\mathcal{G} + \mathcal{H}) \cdot \mathcal{W}_1, \\ P_3 &= \mathcal{F} \cdot (\mathcal{W}_0 - \mathcal{W}_2), & P_6 &= (\mathcal{F} + 32\mathcal{G} + 32\mathcal{H}) \cdot \mathcal{W}_2, \end{aligned} \quad (9)$$

and the result is assembled as

$$\begin{bmatrix} \gamma_0, \gamma_1, \gamma_2, \gamma_3 \\ \gamma_4, \gamma_5, \gamma_6, \gamma_7 \\ \gamma_8, \gamma_9, \gamma_{10}, \gamma_{11} \end{bmatrix} = \begin{bmatrix} P_3 + P_4 + P_6 \\ P_2 - P_4 + P_5 \\ P_1 - P_2 - P_3 \end{bmatrix}. \quad (10)$$

The products P_2 , P_3 , and P_4 use the base matrices $\mathcal{H}$, $\mathcal{F}$, and $32\mathcal{G}$ directly. The products P_1 , P_5 , and P_6 require combined matrices, precomputed by element-wise addition of 4×4 Toeplitz blocks. Since the sum of Toeplitz matrices is again Toeplitz, each combined matrix is determined by its seven parameters. The multiplication by $32 = 2^5$ is implemented as a left shift by 5 bits.

Combined Matrices. The combined matrix for P_1 is $\mathcal{F} + \mathcal{G} + \mathcal{H}$:

$$\begin{bmatrix} \alpha_0 + \alpha_8 + \alpha_4 & 32\alpha_{11} + \alpha_7 + \alpha_3 & 32\alpha_{10} + \alpha_6 + \alpha_2 & 32\alpha_9 + \alpha_5 + \alpha_1 \\ \alpha_1 + \alpha_9 + \alpha_5 & \alpha_0 + \alpha_8 + \alpha_4 & 32\alpha_{11} + \alpha_7 + \alpha_3 & 32\alpha_{10} + \alpha_6 + \alpha_2 \\ \alpha_2 + \alpha_{10} + \alpha_6 & \alpha_1 + \alpha_9 + \alpha_5 & \alpha_0 + \alpha_8 + \alpha_4 & 32\alpha_{11} + \alpha_7 + \alpha_3 \\ \alpha_3 + \alpha_{11} + \alpha_7 & \alpha_2 + \alpha_{10} + \alpha_6 & \alpha_1 + \alpha_9 + \alpha_5 & \alpha_0 + \alpha_8 + \alpha_4 \end{bmatrix}$$

The combined matrix for P_5 is $\mathcal{F} + 32\mathcal{G} + \mathcal{H}$:

$$\begin{bmatrix} \alpha_0 + 32\alpha_8 + \alpha_4 & 32\alpha_{11} + 32\alpha_7 + \alpha_3 & 32\alpha_{10} + 32\alpha_6 + \alpha_2 & 32\alpha_9 + 32\alpha_5 + \alpha_1 \\ \alpha_1 + 32\alpha_9 + \alpha_5 & \alpha_0 + 32\alpha_8 + \alpha_4 & 32\alpha_{11} + 32\alpha_7 + \alpha_3 & 32\alpha_{10} + 32\alpha_6 + \alpha_2 \\ \alpha_2 + 32\alpha_{10} + \alpha_6 & \alpha_1 + 32\alpha_9 + \alpha_5 & \alpha_0 + 32\alpha_8 + \alpha_4 & 32\alpha_{11} + 32\alpha_7 + \alpha_3 \\ \alpha_3 + 32\alpha_{11} + \alpha_7 & \alpha_2 + 32\alpha_{10} + \alpha_6 & \alpha_1 + 32\alpha_9 + \alpha_5 & \alpha_0 + 32\alpha_8 + \alpha_4 \end{bmatrix}$$

The combined matrix for P_6 is $\mathcal{F} + 32\mathcal{G} + 32\mathcal{H}$:

$$\begin{bmatrix} \alpha_0 + 32\alpha_8 + 32\alpha_4 & 32\alpha_{11} + 32\alpha_7 + 32\alpha_3 & 32\alpha_{10} + 32\alpha_6 + 32\alpha_2 & 32\alpha_9 + 32\alpha_5 + 32\alpha_1 \\ \alpha_1 + 32\alpha_9 + 32\alpha_5 & \alpha_0 + 32\alpha_8 + 32\alpha_4 & 32\alpha_{11} + 32\alpha_7 + 32\alpha_3 & 32\alpha_{10} + 32\alpha_6 + 32\alpha_2 \\ \alpha_2 + 32\alpha_{10} + 32\alpha_6 & \alpha_1 + 32\alpha_9 + 32\alpha_5 & \alpha_0 + 32\alpha_8 + 32\alpha_4 & 32\alpha_{11} + 32\alpha_7 + 32\alpha_3 \\ \alpha_3 + 32\alpha_{11} + 32\alpha_7 & \alpha_2 + 32\alpha_{10} + 32\alpha_6 & \alpha_1 + 32\alpha_9 + 32\alpha_5 & \alpha_0 + 32\alpha_8 + 32\alpha_4 \end{bmatrix}$$

Inner TMVP and Cost. Each of the six products $P_1, \dots, P_6$ is a 4×4 TMVP, computed via the two-level recursive decomposition at $9\mathbf{M} + 17\mathbf{A} + 10\mathbf{A}_d$ per product, yielding $54\mathbf{M} + 102\mathbf{A} + 60\mathbf{A}_d$ for all six. The vector differences $\mathcal{W}_0 - \mathcal{W}_1$, $\mathcal{W}_0 - \mathcal{W}_2$, and $\mathcal{W}_1 - \mathcal{W}_2$ contribute $12\mathbf{A}$, the three combined matrix precomputations contribute $42\mathbf{A}$, and the final result assembly (10) adds $24\mathbf{A}_d$. The overall cost is $54\mathbf{M} + 156\mathbf{A} + 84\mathbf{A}_d$, plus 11 shifts for precomputing the scaled values $32\alpha_1, \dots, 32\alpha_{11}$.

4 Implementation and Performance

All implementations are written in portable C based on the SUPERCOP framework [BL09], without platform-specific assembly or intrinsics. Benchmarks are conducted on two platforms: an Apple M1 Pro (ARM64, 3.228 GHz, macOS, Apple Clang 17) and an Intel Core i7-8565U (x86-64, 1.8 GHz, Linux, GCC with `-march=native`). On the x86-64 platform, the CPU frequency governor is set to `performance` mode, Turbo Boost is disabled, and the benchmark process is pinned to a single core to eliminate scheduling and frequency-scaling variability. The ARM64 platform uses a unified performance architecture without dynamic frequency scaling. Cycle counts are obtained via `mach_absolute_time()`

Table 1: Cycle counts on ARM64 (Apple M1 Pro, 3.228 GHz) and x86-64 (Intel Core i7-8565U, 1.8 GHz).

	Method	ARM64 (Apple M1 Pro)				x86-64 (Intel i7-8565U)			
		mul	inv	smul_base	smul	mul	inv	smul_base	smul
<i>Curve5453</i>	tmvp ₁₀₋₅₅	134.02	79,383	871,188	871,898	259.6	158,866	1,909,313	1,837,785
	tmvpsb ₁₀₋₅₅	157.92	84,102	934,246	933,228	260.8	158,713	1,862,613	1,769,136
	ref ₁₀₋₅₄₅	138.82	78,250	900,358	914,670	265.4	162,985	1,901,102	1,903,587
	sbtmvp ₉₋₆₁	112.47	65,588	752,762	755,616	257.8	142,672	1,426,723	1,674,139
	sb ₉₋₆₁	121.13	75,848	801,418	804,899	259.3	152,052	1,696,459	1,734,208
<i>Curve6071</i>	tmvp ₁₀₋₆₁	123.75	78,051	881,874	903,542	210.8	135,982	1,496,073	1,497,513
	tmvpsb ₁₀₋₆₁	135.35	85,928	968,776	965,012	230.6	148,680	1,634,995	1,637,029
	sbtmvp _{3₁₂₋₅₁}	162.42	105,302	1,170,898	1,172,017	276.7	178,416	1,961,993	1,964,547
	sbtmvp _{2₁₂₋₅₁}	158.42	98,218	1,102,103	1,107,726	269.9	173,842	1,911,685	1,922,936
	sbtmvp _{4₁₂₋₅₁}	134.33	87,065	988,101	989,259	228.8	147,409	1,621,019	1,623,045
	tmvp ₁₂₋₅₁	111.96	74,050	853,338	893,856	190.7	119,452	1,528,514	1,530,412

on ARM64 and RDTSCP with serializing barriers on x86-64; we report the median over multiple measurement rounds.

Each implementation is identified by the convention `methodn-r`, where the base name indicates the multiplication strategy, n is the number of limbs, and r is the radix exponent (or the field bit-length for the schoolbook reference). For $\mathbb{F}_{5453}$, the strategies developed in Section 3 are `sbtmvp9-61` (60M, schoolbook outer with size-3 TMVP inner products) and `sb9-61` (81M, schoolbook at all levels). For $\mathbb{F}_{6071}$, Section 3 develops four strategies over the 12-limb radix-2⁵¹ representation: `sbtmvp312-51` (104M), `sbtmvp412-51` (81M), a non-recursive variant `sbtmvp212-51` (schoolbook at the 2×2 block level), and the block-level TMVP method `tmvp12-51` (54M). As reference, we include the 10-limb TMVP-based methods from [COKTY25]: `tmvp10-55` (77M) and `tmvpsb10-55` (95M) for $\mathbb{F}_{5453}$, along with the schoolbook method `ref10-545` (101M); and `tmvp10-61` (77M) and `tmvpsb10-61` (95M) for $\mathbb{F}_{6071}$. Other classical strategies from [COKTY25], such as `donna` and `kummer`, are omitted from the tables.

Four operations are benchmarked: field multiplication (mul), field inversion (inv), fixed-base scalar multiplication (smul_base), and variable-base scalar multiplication (smul). Field addition, subtraction, and inversion follow the implementations described in [COKTY25]; only the multiplication routine differs across strategies. Scalar multiplication uses the Montgomery ladder [BL17], which processes scalar bits in constant time using x -coordinate-only arithmetic. Inversion is computed via Fermat’s Little Theorem using optimized addition chains from [NS18]. Table 1 reports cycle counts on both platforms.

For $\mathbb{F}_{5453}$, the 9-limb method `sbtmvp9-61` achieves the best results across all metrics on both platforms. On ARM64, it reaches 112 cycles for field multiplication, a 16% reduction compared to `tmvp10-55` (134 cycles), consistent with the decrease from 77M to 60M. The schoolbook variant `sb9-61` (81M) also outperforms the 10-limb TMVP (77M), indicating that the structural advantage of fewer limbs and reduced carry propagation outweighs the difference in multiplication count. On x86-64, the multiplication cycle counts converge more tightly across methods, but `sbtmvp9-61` maintains a clear advantage at the scalar multiplication level.

For $\mathbb{F}_{6071}$, `tmvp12-51` achieves the fastest field multiplication on both platforms: 112 cycles on ARM64 (9.5% faster than `tmvp10-61`) and 191 cycles on x86-64 (9.5% faster). On x86-64, this advantage extends uniformly to all metrics. On ARM64, however, `tmvp10-61` is competitive for variable-base scalar multiplication, as the 12-limb representation incurs additional carry propagation per field operation, and this overhead accumulates across the 607 ladder iterations. Among the intermediate strategies, `sbtmvp312-51` (104M) performs worse than the 10-limb baseline on both platforms, confirming that the finer block partition is suboptimal. The method `sbtmvp412-51` (81M), by contrast, is competitive with `tmvpsb10-61` (95M) despite the carry overhead of 12 limbs, reflecting the practical efficiency

Table 2: ECDH throughput on ARM64 (Apple M1 Pro).

Curve	Security	Impl.	ops/s	$\mu\text{s/op}$
X448	224	OpenSSL	7,597	132
Brainpool P-512	256	OpenSSL	1,243	805
NIST P-521	259	OpenSSL	4,705	213
<i>Curve5453</i> (<i>sbtmvp</i> ₉₋₆₁)	271	This work	4,262	235
<i>Curve6071</i> (<i>tmvp</i> ₁₂₋₅₁)	302	This work	3,803	263

of the 3×3 block decomposition.

Among SafeCurves-compliant curves [BL25] at security levels beyond 200 bits, M-511 [ABPR13] (254-bit security, $p = 2^{511} - 187$), E-521 [GS15] (259-bit, $p = 2^{521} - 1$), and Ed448-Goldilocks [Ham15] (224-bit) represent the closest prior work. Curve5453 and Curve6071, introduced in [COKTY25], extend this family to 271 and 302 bits of classical security, respectively. Although direct comparison across platforms and implementation languages is limited, the results provide useful context: *sbtmvp*₉₋₆₁ achieves 755,616 cycles for variable-base scalar multiplication on ARM64, compared to 943,000 cycles reported for E-521 on a different microarchitecture (Intel Haswell) [GS15] using hand-optimized assembly, while providing 12 additional bits of security with portable C code. *tmvp*₁₂₋₅₁ achieves 893,856 cycles on ARM64 with 302-bit security—43 bits beyond E-521. The standardized curves at comparable security levels, NIST P-521 and Brainpool P-512 [Bra05], do not satisfy several SafeCurves criteria [BL25] including twist security; the Montgomery form of the proposed curves provides inherent constant-time x -only arithmetic.

To provide a controlled same-platform comparison, Table 2 reports ECDH throughput for the proposed curves alongside OpenSSL implementations of standardized curves, all measured on the Apple M1 Pro. The proposed implementations replicate the measurement methodology of `openssl speed`, counting operations over a fixed 10-second interval; OpenSSL 3.6.0, compiled with platform-optimized ARM64 assembly, is used for X448, NIST P-521, and Brainpool P-512.

Curve5453 achieves 4,262 ECDH operations per second with *sbtmvp*₉₋₆₁, reaching 90.6% of OpenSSL’s NIST P-521 throughput (4,705 ops/s) while providing 12 additional bits of security with portable C code rather than hand-optimized assembly. Curve6071 achieves 3,803 ops/s with *tmvp*₁₂₋₅₁, corresponding to 80.8% of P-521’s throughput with 43 additional bits of security. Both curves outperform Brainpool P-512 by a factor of 3.1–3.4 $\times$, illustrating the performance cost of random primes lacking arithmetic structure amenable to TMVP-based multiplication.

5 Conclusion

We have shown that alternative radix representations for the fields underlying Curve5453 and Curve6071 yield substantially more efficient TMVP-based multiplication than the 10-limb baseline of [COKTY25]. For $\mathbb{F}_{5453}$, the 9-limb radix-2⁶¹ representation reduces the multiplication cost from 77M to 60M (22%), owing to a 3×3 block partition whose sub-matrices align directly with the size-3 TMVP formula. For $\mathbb{F}_{6071}$, the 12-limb radix-2⁵¹ representation enables a hierarchical 3×3 block TMVP that achieves 54M (30% reduction). Dual-platform benchmarks confirm that these algebraic savings translate into practical speedups: *sbtmvp*₉₋₆₁ delivers 16% faster field multiplication and 13% faster scalar multiplication on ARM64 compared to the 10-limb TMVP, while *tmvp*₁₂₋₅₁ achieves 9.5% faster field multiplication on both platforms. An ECDH throughput comparison against OpenSSL on the same platform shows that Curve5453 reaches 90.6% of NIST P-521’s throughput with 12 additional bits of security, and Curve6071 provides 302-bit security at 80.8% of P-521’s throughput—both using portable C rather than hand-optimized assembly.

References

- [ABPR13] Diego F. Aranha, Paulo S. L. M. Barreto, Geovandro C. C. F. Pereira, and Jefferson E. Ricardini. A note on high-security general-purpose elliptic curves. Cryptology ePrint Archive, Paper 2013/647, 2013. <https://eprint.iacr.org/2013/647>.
- [AC18] Shoukat Ali and Murat Cenk. Faster residue multiplication modulo 521-bit mersenne prime and an application to ecc. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 65(8):2477–2490, 2018.
- [Ber06] Daniel J. Bernstein. Curve25519: New diffie-hellman speed records. In *PKC*, volume 3958 of *LNCS*, pages 207–228. Springer, 2006.
- [BL09] Daniel J Bernstein and Tanja Lange. Supercop: System for unified performance evaluation related to cryptographic operations and primitives, 2009.
- [BL17] D.J. Bernstein and T. Lange. *Montgomery curves and the Montgomery ladder*. Cryptology ePrint Archive. International Association for Cryptologic Research, 2017.
- [BL25] Daniel J Bernstein and Tanja Lange. Safe curves for elliptic-curve cryptography. *Information Security in a Connected World: Celebrating the Life and Work of Ed Dawson*, pages 124–191, 2025.
- [Bra05] ECC Brainpool. Ecc brainpool standard curves and curve generation, 2005.
- [COKTY25] Murat Cenk, N. Gamze Orhon Kılıç, Halil Kemal Taşkın, and Oğuz Yayla. Secure Montgomery curves over TMVP-friendly primes for high-performance ECC. Cryptology ePrint Archive, 2025.
- [Cra92] Richard E Crandall. Method and apparatus for public key exchange in a cryptographic system. *US Patent 5,159,632*, 1992.
- [CS18] Craig Costello and Benjamin Smith. Montgomery curves and their arithmetic: The case of large characteristic fields. *Journal of Cryptographic Engineering*, 8(3):227–240, 2018.
- [GS15] Robert Granger and Michael Scott. Faster ecc over f2521-1. In Jonathan Katz, editor, *Public-Key Cryptography – PKC 2015*, pages 539–553, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
- [Ham15] Mike Hamburg. Ed448-goldilocks, a new elliptic curve. Cryptology ePrint Archive, Paper 2015/625, 2015. <https://eprint.iacr.org/2015/625>.
- [HJN⁺20] Thomas Häner, Samuel Jaques, Michael Naehrig, Martin Roetteler, and Mathias Soeken. Improved quantum circuits for elliptic curve discrete logarithms. In *International conference on post-quantum cryptography*, pages 425–444. Springer, 2020.
- [Kob87] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of computation*, 48(177):203–209, 1987.
- [Mil86] Victor S. Miller. Use of elliptic curves in cryptography. In Hugh C. Williams, editor, *Advances in Cryptology — CRYPTO ’85 Proceedings*, pages 417–426, Berlin, Heidelberg, 1986. Springer Berlin Heidelberg.
- [Mon87] Peter L Montgomery. Speeding the pollard and elliptic curve methods of factorization. *Mathematics of computation*, 48(177):243–264, 1987.

- [Nat23] National Institute of Standards and Technology (NIST). Digital Signature Standard (DSS). Federal Information Processing Standards Publication FIPS PUB 186-5, National Institute of Standards and Technology (NIST), February 2023.
- [NS18] Kaushik Nath and Palash Sarkar. Efficient arithmetic in (pseudo-) mersenne prime order fields. *Cryptology ePrint Archive*, 2018.
- [Sho94] P.W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994.
- [TC18] Halil Kemal Taşkın and Murat Cenk. Speeding up curve25519 using toeplitz matrix-vector multiplication. In *Proceedings of the Fifth Workshop on Cryptography and Security in Computing Systems*, pages 1–6, 2018.
- [TC20] Halil Kemal Taşkın and Murat Cenk. Tmvp-friendly primes for efficient elliptic curve cryptography. In *2020 International Conference on Information Security and Cryptology (ISCTURKEY)*, pages 80–87. IEEE, 2020.